

C++ Inheritance

In C++, inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In such way, you can reuse, extend or modify the attributes and behaviors which are defined in other class.

In C++, the class which inherits the members of another class is called derived class and the class whose members are inherited is called base class. The derived class is the specialized class for the base class.

Syntax of Inheritance:

```
class derived_class_name : access-specifier base_class_name
{
    // body ....
};
```

where,

- **class:** keyword to create a new class
- **derived_class_name:** name of the new class, which will inherit the base class
- **access-specifier:** Specifies the access mode which can be either of private, public or protected. If neither is specified, private is taken as default.
- **base-class-name:** name of the base class.

Example:

```
#include <iostream>
using namespace std;

class Person
{
public:
    string name;
    int age;
```

```
Person()
{
    cout << "Parent constructor called..\n";
};

class Student : public Person
{
public:
    int rollNumber;

    Student()
    {
        cout << "Child constructor called..\n";
    }

    void getInfo()
    {
        cout << "Name : " << name << endl;
        cout << "Age : " << age << endl;
        cout << "RollNumber : " << rollNumber << endl;
    }
};

int main()
{
    Student s1;
    s1.name = "Aadil";
    s1.age = 20;
    s1.rollNumber = 1234;
    s1.getInfo();

    return 0;
}
```

When we have parameterized constructor in base class:

```
#include <iostream>
using namespace std;

class Person{
public:
    string name;
    int age;

    Person(string name, int age) {
        cout << "Parent constructor called..\n";
        this->name = name;
        this->age = age;
    };
};

class Student : public Person
{
public:
    int rollNumber;

    Student(string name, int age, int rollNumber) : Person(name, age)
    {
        cout << "Child constructor called..\n";
        this->rollNumber = rollNumber;
    }

    void getInfo()
    {
        cout << "Name : " << name << endl;
```

```
    cout << "Age : " << age << endl;
    cout << "RollNumber : " << rollNumber << endl;
}
};

int main()
{
    Student s1("Aadil", 20, 1234);
    s1.getInfo();

    return 0;
}
```

Advantage of C++ Inheritance

Code reusability: Now you can reuse the members of your parent class. So, there is no need to define the member again. So less code is required in the class.

C++ supports five types of inheritance:

- Single inheritance
- Multiple inheritance
- Hierarchical inheritance
- Multilevel inheritance
- Hybrid inheritance

Base Class

A base class (also called a parent class or superclass) is a class whose properties and methods are inherited by other classes. It provides common features that other classes can use.

```
class Person {  
public:  
    string name;  
    int age;  
};
```

Derived Class

A derived class (also called a child class or subclass) is a class that inherits properties and methods from a base class. It can also have additional properties and methods or override the behavior of the base class.

Syntax:

```
// syntax  
class className : AccessSpecifier BaseClassName {  
  
    // body of derived class  
};
```

Example:

```
class Student : public Person  
{  
public:  
    int rollNumber;  
};
```

Access Specifiers:

Access specifiers in C++ play a crucial role in inheritance by controlling how the members (attributes and methods) of the base class are accessed in the derived class.

There are 3 types of access specifiers used in inheritance:

1. Public Inheritance

When a class inherits from a base class using public inheritance, all the public members of the base class remain public in the derived class, and all protected members of the base class remain protected in the derived class.

```
class Base {
public:
    int publicVar;
protected:
    int protectedVar;
private:
    int privateVar;
};

class Derived : public Base {
public:
    void display() {
        publicVar = 1; // OK: public member
        protectedVar = 2; // OK: protected member
        // privateVar = 3; // Error: private member not accessible
    }
};
```

2. Protected Inheritance

When a class inherits from a base class using protected inheritance, all the public and protected members of the base class become protected in the derived class. This means they are accessible within the derived class and by classes derived from the derived class but not from outside.

```
class Base {
public:
    int publicVar;
protected:
    int protectedVar;
private:
    int privateVar;
};

class Derived : protected Base {
public:
    void display() {
        publicVar = 1; // OK: now a protected member
        protectedVar = 2; // OK: protected member
        // privateVar = 3; // Error: private member not accessible
    }
};
```

3. Private Inheritance

When a class inherits from a base class using private inheritance, all the public and protected members of the base class become private in the derived class. They are only accessible within the derived class and are not accessible to any other class, including further derived classes.

```

class Base
{
public:
    int publicVar;

protected:
    int protectedVar;

private:
    int privateVar;
};

class Derived : private Base
{
public:
    void display()
    {
        publicVar = 1; // OK: now a private member
        protectedVar = 2; // OK: now a private member
        // privateVar = 3; // Error: private member not accessible
    }
};

```

Types of Inheritance:

1. Single Inheritance

In single inheritance, a class is allowed to inherit from only one class. i.e. one base class is inherited by one derived class only.

```

class subclass_name : access_mode base_class {
    // body of subclass

```



```
};
```

Example:

```
#include <iostream>
using namespace std;

class Person
{
public:
    string name;
    int age;

    Person()
    {
        cout << "Parent constructor called..\n";
    }
};

class Student : public Person
{
public:
    int rollNumber;

    Student()
    {
        cout << "Child constructor called..\n";
    }

    void getInfo()
    {
        cout << "Name : " << name << endl;
        cout << "Age : " << age << endl;
        cout << "RollNumber : " << rollNumber << endl;
    }
}
```

```
};

int main()
{
    Student s1;
    s1.name = "Aadil";
    s1.age = 20;
    s1.rollNumber = 1234;
    s1.getInfo();

    return 0; }
```

2. Multiple Inheritance

Multiple Inheritance is a feature of C++ where a class can inherit from more than one class. i.e one **subclass** is inherited from more than one **base class**.

Syntax:

```
class subclass_name : access_mode base_class1, access_mode base_class2, ...
{
    // body of subclass
};
```

Example:

```
#include <iostream>
using namespace std;

class Student
{
public:
    string name;
```

```
    int rollNumber;
};

class Teacher
{
public:
    string subject;
    double salary;
};

class TeachingAssistant : public Student, public Teacher
{
};

int main()
{
    TeachingAssistant t1;
    t1.name = "Tony stark";
    t1.subject = "CS";
    cout << t1.name << endl;
    cout << t1.subject << endl;
    return 0;
}
```

3. Multilevel Inheritance

In this type of inheritance, Multilevel inheritance involves a chain of inheritance where a class is derived from another derived class.

```
#include <iostream>
using namespace std;
```

```
// Parent class
class Person
{
public:
    string name;
    int age;
};

// Parent class
class Student : public Person {
public:
    int rollNumber;
};

// Child class
class GradStudent : public Student
{
public:
    string researchArea;
};

int main(){
    GradStudent s1;
    s1.name = "Tony stark";
    s1.researchArea = "Quantum Physics";

    cout << s1.name << endl;
    cout << s1.researchArea << endl;

    return 0;
}
```

4. Hierarchical Inheritance

In this type of inheritance, more than one subclass is inherited from a single base class.
i.e. more than one derived class is created from a single base class.

```
#include <iostream>
using namespace std;

class Person
{
public:
    string name;
    int age;
};

class Student : public Person
{
public:
    int rollNumber;
};

class Teacher : public Person {

public:
    string subject;
};

int main() {
    Student s1;
    s1.name = "Tony";
    cout << s1.name << endl;

    return 0;
}
```

5. Hybrid Inheritance

Hybrid Inheritance is implemented by combining more than one type of inheritance. For example: Combining Hierarchical inheritance and Multiple Inheritance will create hybrid inheritance in C++

There is no particular syntax of hybrid inheritance. We can just combine two of the above inheritance types.

Type conversion :

Type conversion in inheritance, also known as type casting, is a process of converting an object of one type to another within an inheritance hierarchy. In C++, type conversions are crucial when dealing with polymorphism, allowing a program to treat an object of a derived class as an object of its base class or vice versa. Here are the main types of type conversions in inheritance:

1. Upcasting

Upcasting is the process of converting a pointer of a derived class to a pointer of its base class. This type of conversion is always safe and implicit

```
class Base{
public:
    void display(){
        cout << "Base class display function" << endl;
    }
};

class Derived : public Base {
public:
    void show(){
        cout << "Derived class show function" << endl;
    }
};
```

```
int main(){
    Derived derivedObj;
    Base *basePtr = &derivedObj; // Implicit upcasting
    basePtr->display();    // Calls Base class function
    return 0;}
```

2. Downcasting

Downcasting is the process of converting a pointer of a base class to a pointer of a derived class. This type of conversion is potentially unsafe and usually requires an explicit cast. Downcasting is typically checked at runtime using mechanisms like `dynamic_cast` to ensure the cast is valid.

```
class Parent {
public:
    void base(){
        cout<<" It's Parent class function " << endl;
    }
};

class Child : public Parent
{
public:
    void derive(){
        cout <<" it's Child class function " << endl;
    }
};

int main(){
    Parent pobj; // create Parent's object
    Child *cobj; // create Child's object
```

```
// explicit type cast is required in downcasting
cobj = (Child *)&pobj;
cobj->derive();

return 0;
```

Abstract Class:

An abstract class in C++ is a class that cannot be instantiated (Abstract classes cannot be used to create objects directly). on its own and is typically used as a base class. Abstract classes are designed to be inherited by other classes and at least one function in the class should be declared as a pure virtual function using the = 0 syntax.

A pure virtual function is a function declared in the base class that has no definition within that class and must be overridden by derived classes.

Abstract classes are typically used to define an interface for derived classes. Or we can say they are used to define the blueprint for other classes (derived classes).

Example:

```
#include <iostream>
using namespace std;

class Shape
{
    virtual void draw() = 0; // pure virtual function
};

class Circle : public Shape
{
public:
    void draw()
    {
```



```
    cout << "Drawing a circle\n";  
}  
};  
  
int main()  
{  
    Circle c1;  
    c1.draw();  
    return 0;  
}
```

Virtual Base Class:

In C++, a virtual base class is used in the context of multiple inheritance to solve the "diamond problem," which occurs when a class inherits from two classes that have a common base class.

This common base class ends up being included twice, once through each of the parent classes, which can lead to ambiguity and redundancy.

By declaring a base class as a virtual base class, you ensure that only one instance of that base class is shared among all the derived classes.

Example:

```
class Base {  
public:  
    void display(){  
        cout << "Base class display function" << endl;  
    }  
};
```

```

class Derived1 : virtual public Base {};

class Derived2 : virtual public Base{};

class Final : public Derived1, public Derived2 {};

int main(){
    Final obj;
    obj.display(); // No ambiguity, only one Base instance
    return 0;
}

```

Operator Overloading:

C++ provides a special function to change the current functionality of some operators within its class which is often called as operator overloading. Operator Overloading is the method by which we can change the function of some specific operators to do some different tasks.

Types of Operator Overloading:

Unary Operator Overloading:

In **C++**, unary operator overloading involves operators like +, -, !, &, *, ++, --, which operate on a single operand.

A unary operator is an operator that operates on a single operand. These operators are used to perform operations on a single operand, such as changing the sign of a number or incrementing/decrementing its value.

Syntax:

```

class className {

```

```

public:
    returnType operator<symbol>() {
        // custom behaviour for the operator
    }
};

```

Example:

```

class Increase {
private:
    int value;
public:
    Increase(int n) {
        value = n;
    }
    void operator++() {
        value += 5;
    }
    void display() {
        cout << "value is : " << value << endl;
    }
};

int main() {
    Increase Inc(10);
    ++Inc;
    Inc.display();
}

```

Binary Operator Overloading:

Binary operator overloading in C++ allows you to define custom behavior for binary operators (operators that take two operands) when they are applied to objects of a class. This process lets you extend the functionality of operators such as +, -, *, /, and others to work with user-defined types in addition to built-in types.

Syntax:

```
return_type operator op(argument) {  
    // Function Body  
}
```

Example:

```
class Data  
{  
public:  
    int num;  
  
    Data(int n)  
    {  
        num = n;  
    }  
  
    Data operator+(Data d2) // Data d2 is 2nd operand  
    {  
        Data temp(0); // temp object to store d1+d2  
  
        temp.num = num + d2.num; // temp=30+10  
        return temp;  
    }  
    void display()  
    {  
        cout << "Value = " << num << endl;  
    }  
};  
int main()  
{  
  
    Data d1(30), d2(10), result(0);  
    result = d1 + d2;  
    result.display();  
    return 0;  
}
```

